Bookstore Sales and Ratings Analysis Using SQL

Project Overview

The competitive bookstore industry relies heavily on operational insights and reader engagement metrics to drive growth. This project involves a comprehensive analysis of book sales and rating data to explore key performance indicators, author contribution, and financial efficiency. With changing reader preferences and market trends, it is crucial to assess how various factors—such as author reputation, publication year, and reader ratings—influence commercial success and inventory management.

Objective:

This project analyzes book, rating, and sales data to:

- **Identify top-performing authors** and their contribution to the store's inventory.
- **Evaluate financial performance** by calculating profit margins across different titles.
- **Understand reader engagement** through rating counts and average ratings.
- **Discover historical trends** in book performance based on publication years.
- **Recommend actionable insights** to optimize inventory decisions and boost high-rated titles.

Why This Project?

- **Data-Driven Decision Making:** Understanding the relationship between book quality (ratings) and quantity (sales) helps in making informed inventory and marketing decisions.
- **Performance Bottlenecks:** This project helps uncover which books or authors are underperforming, enabling the bookstore to refine its strategy.

- **Business Growth:** By analyzing profit margins and reader sentiment, the bookstore can focus on high-impact titles, ultimately improving overall profitability and customer satisfaction.

Key Queries and Insights :

1. Retrieve All Book Records:

Input:

```
SELECT * FROM books;
```

Output:

book_id	book_name	author	genre	publish_year	price	sales_units
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	1925	10.99	5000
2	1984	George Orwell	Dystopian	1949	8.99	7000
3	To Kill a Mockingbird	Harper Lee	Fiction	1960	12.50	8000
4	The Hobbit	J.R.R. Tolkien	Fantasy	1937	15.00	6000
5	Pride and Prejudice	Jane Austen	Romance	1813	7.99	4500
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Insight: "This query retrieves the complete dataset, providing a fundamental overview of the book catalog's structure and metadata."

2. Retrieve All Rating and Performance Records:

Input:

```
SELECT * FROM ratings;
```

Output:

rating_id	book_id	author_rating	book_rating	book_ratings_count	gross_sales	publisher_revenue	sale_price	units_sold
1	1	4.50	4.20	1000	50000.00	20000.00	10.99	5000
2	2	4.80	4.60	1500	60000.00	25000.00	8.99	7000
3	3	4.20	4.10	1200	45000.00	18000.00	12.50	8000
4	4	4.70	4.50	900	40000.00	15000.00	15.00	6000
5	5	4.00	3.90	800	35000.00	12000.00	7.99	4500
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Insight: This query provides a comprehensive overview of the ratings table, which includes essential performance metrics such as author/book ratings, gross sales, publisher revenue, and total units sold for each book.

3. Analyze Database Schema (ratings table):

Input:

```
DESCRIBE ratings;
```

Output:

	Field	Type	Null	Key	Default	Extra
▶	rating_id	int	NO	PRI	NULL	auto_increment
	book_id	int	YES	MUL	NULL	
	author_rating	decimal(3,2)	YES		NULL	
	book_rating	decimal(3,2)	YES		NULL	
	book_ratings_count	int	YES		NULL	
	gross_sales	decimal(10,2)	YES		NULL	
	publisher_revenue	decimal(10,2)	YES		NULL	
	sale_price	decimal(10,2)	YES		NULL	
	units_sold	int	YES		NULL	

Insight: This query provides the structural definition of the ratings table, detailing key fields such as book_id, author_rating, book_rating, gross_sales, and units_sold, which are essential for performing advanced analytical queries.

4. Analyze Book Performance by Sales and Ratings:

Input:

```
1 •   SELECT b.book_name, r.book_rating, r.gross_sales
2     FROM books b
3     JOIN ratings r ON b.book_id = r.book_id;
```

Output:

	book_name	book_rating	gross_sales
▶	The Great Gatsby	4.20	50000.00
	The Great Gatsby	4.20	50000.00
	1984	4.60	60000.00
	1984	4.60	60000.00
	To Kill a Mockingbird	4.10	45000.00
	To Kill a Mockingbird	4.10	45000.00
	The Hobbit	4.50	40000.00
	The Hobbit	4.50	40000.00
	Pride and Prejudice	3.90	35000.00
	Pride and Prejudice	3.90	35000.00

Insight: This query demonstrates the relationship between a book's average rating and its commercial performance (gross sales), helping to identify which high-rated titles are also driving significant revenue.

5. Analyze Book Metadata and Rating Performance:

Input:

```
1 •   SELECT b.book_name, b.publish_year, r.book_rating
2     FROM books b
3     JOIN ratings r ON b.book_id = r.book_id;
```

Output:

book_name	publish_year	book_rating
The Great Gatsby	1925	4.20
The Great Gatsby	1925	4.20
1984	1949	4.60
1984	1949	4.60
To Kill a Mockingbird	1960	4.10
To Kill a Mockingbird	1960	4.10
The Hobbit	1937	4.50
The Hobbit	1937	4.50
Pride and Prejudice	1813	3.90
Pride and Prejudice	1813	3.90

Insight: This query effectively correlates book metadata (title and publication year) with performance metrics (average rating), enabling a time-based analysis of how books from different periods are perceived by readers.

6. Identify Prolific Authors with Performance Metrics:

Input:

```
1  SELECT author, book_count, avg_rating
2  FROM (
3      SELECT
4          b.author,
5          COUNT(b.book_id) AS book_count,
6          AVG(r.author_rating) AS avg_rating
7      FROM books b
8      JOIN ratings r ON b.book_id = r.book_id
9      GROUP BY b.author
10 ) AS AuthorStats
11 WHERE book_count > 1;
```

Output:

author	book_count	avg_rating
F. Scott Fitzgerald	2	4.500000
George Orwell	2	4.800000
Harper Lee	2	4.200000
J.R.R. Tolkien	2	4.700000
Jane Austen	2	4.000000

Insight: This subquery identifies "prolific authors" (those with more than one book in the collection) and calculates their average author rating, which helps in assessing the consistency and quality of an author's contribution to the bookstore's inventory.

7. Identify "Hidden Gems" (High Rating, Moderate Sales):

Input:

```
1 •  SELECT b.book_name, r.book_rating, r.units_sold
2    FROM books b
3    JOIN ratings r ON b.book_id = r.book_id
4    WHERE r.book_rating > 3.0 AND r.units_sold < 5000;
```

Output:

	book_name	book_rating	units_sold
▶	Pride and Prejudice	3.90	4500
	Pride and Prejudice	3.90	4500

Insight: "This query effectively filters for "hidden gems"—books that have achieved high reader approval ratings while maintaining moderate sales volume, suggesting potential for further marketing to reach a wider audience."

8. Book Performance Overview (Rating vs. Sales):

Input:

```sql
SELECT
    b.book_name,
    r.book_rating,
    r.units_sold
FROM books b
JOIN ratings r ON b.book_id = r.book_id;
```

Output:

book_name	book_rating	units_sold
The Great Gatsby	4.20	5000
The Great Gatsby	4.20	5000
1984	4.60	7000
1984	4.60	7000
To Kill a Mockingbird	4.10	8000
To Kill a Mockingbird	4.10	8000
The Hobbit	4.50	6000
The Hobbit	4.50	6000
Pride and Prejudice	3.90	4500
Pride and Prejudice	3.90	4500

Insight: This query provides a direct comparison between reader-assigned ratings and actual unit sales, allowing for an evaluation of how consumer sentiment correlates with the commercial success of each book in the catalog.

9. Calculate Profit Margin per Book:

Input:

```
1 •  SELECT b.book_name, (r.gross_sales - r.publisher_revenue) AS profit_margin
2    FROM books b
3    JOIN ratings r ON b.book_id = r.book_id;
```

Output:

	book_name	profit_margin
▶	The Great Gatsby	30000.00
	The Great Gatsby	30000.00
	1984	35000.00
	1984	35000.00
	To Kill a Mockingbird	27000.00
	To Kill a Mockingbird	27000.00
	The Hobbit	25000.00
	The Hobbit	25000.00
	Pride and Prejudice	23000.00
	Pride and Prejudice	23000.00

Insight: This query calculates the profit margin for each book by subtracting publisher revenue from gross sales, highlighting the financial efficiency of individual titles in the bookstore's collection.

10. Identify Most Rated Books:

```
1 •   SELECT b.book_name, r.book_ratings_count
2     FROM books b
3     JOIN ratings r ON b.book_id = r.book_id
4
5     ORDER BY r.book_ratings_count DESC
6     LIMIT 5;
```

Output:

book_name	book_ratings_count
1984	1500
1984	1500
To Kill a Mockingbird	1200
To Kill a Mockingbird	1200
The Great Gatsby	1000

Insight: This query identifies the top five most rated books in the inventory, which highlights the titles with the highest level of reader engagement and community interaction.

11. Analyze Sales and Revenue by Publishing Year:

Input:

```
1 •  SELECT b.publish_year, SUM(r.units_sold) AS total_units, SUM(r.gross_sales) AS total_revenue
2    FROM books b
3    JOIN ratings r ON b.book_id = r.book_id
4    GROUP BY b.publish_year;
```

Output:

	publish_year	total_units	total_revenue
▶	1925	10000	100000.00
	1949	14000	120000.00
	1960	16000	90000.00
	1937	12000	80000.00
	1813	9000	70000.00

Insight: This query groups data by publication year to aggregate total unit sales and revenue, which helps in identifying the historical trends and years with the highest commercial impact for the bookstore's collection.

12. dentify Books Where Author Rating Exceeds Book Rating:

Input:

```
1 •   SELECT b.book_name, r.author_rating, r.book_rating
2     FROM books b
3     JOIN ratings r ON b.book_id = r.book_id
4     WHERE r.author_rating > r.book_rating;
```

Output:

book_name	author_rating	book_rating
The Great Gatsby	4.50	4.20
1984	4.80	4.60
To Kill a Mockingbird	4.20	4.10
The Hobbit	4.70	4.50
Pride and Prejudice	4.00	3.90
The Great Gatsby	4.50	4.20
1984	4.80	4.60
To Kill a Mockingbird	4.20	4.10
The Hobbit	4.70	4.50
Pride and Prejudice	4.00	3.90

Insight: This query highlights instances where the author's overall reputation (author rating) exceeds the specific performance of a particular book, which may indicate that the author's brand value is higher than the reception of that individual title.

1. Key Learnings

- **Relational Database Management:** Learned how to effectively link two separate tables (books and ratings) using JOIN operations to aggregate related data.
- **Data Aggregation:** Gained proficiency in using SQL aggregate functions like SUM, AVG, COUNT, and GROUP BY to extract meaningful summaries from raw datasets.
- **Query Optimization:** Developed skills in using WHERE clauses, ORDER BY, and LIMIT to filter, sort, and refine data analysis efficiently.
- **Subqueries:** Acquired the ability to use subqueries within FROM clauses to handle complex data manipulation tasks.
- **Analytical Thinking:** Improved the ability to interpret raw database output into actionable business insights.

2. Challenges

- **Schema Mapping:** Encountered discrepancies between the column names in the provided documentation and the actual database schema (e.g., book_rating vs book_average_rating), which required utilizing the DESCRIBE command to troubleshoot.
- **Query Logic:** Initially faced difficulties in determining the correct relational keys for joining tables, which was resolved by performing a thorough review of the database design.
- **Data Integrity:** Had to exercise caution with data types (such as DECIMAL vs INTEGER) during mathematical calculations to ensure the accuracy of the profit margin results.